

Test Strategy

Our test strategy was centered on unit testing using the JUnit framework, which integrates seamlessly with Maven. The primary goal was to validate each component of the application in isolation before verifying their integration. This bottom up approach ensures that individual pieces of logic are correct before they are combined, simplifying debugging and increasing confidence in the system as a whole. Our strategy was implemented in two main layers.

Model Unit Testing:

- Objective: To verify the core "business logic" of the application.
- Execution: Each class in the model package (e.g., Student, Course, Session) was tested independently.
- Example: CourseTest validates that the logic for enrollment, waitlisting, and capacity limits functions correctly without needing any user input or file system.

Application Integration Testing:

- Objective: To verify the interaction between the user interface logic (Menu), the application state (Session), and external dependencies (like the Transaction class for file I/O).
- Execution: MenuTest simulates user interaction by providing a mock Scanner with pre-defined inputs.
- Example: The testAddStudentUI test provides fake user input (ID, name, email) and asserts that the Session object is correctly modified. The testSaveAndLoadUI test verifies that the Menu class can correctly trigger file system operations and that the data is

persisted and retrieved accurately, ensuring the test cleans up any generated files afterward.

By testing both the isolated logic (unit tests) and the component interactions (integration tests), we were able to build a comprehensive test suite that validates the application's correctness from the core model up to the user facing application layer.

TERMINAL OUTPUT:

```
[INFO] -----
```

```
[INFO] T E S T S
```

```
[INFO] -----
```

```
[INFO] Running edu.uca.registration.app.MenuTest
```

```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.224 s -- in  
edu.uca.registration.app.MenuTest
```

```
[INFO] Running edu.uca.registration.model.CourseTest
```

```
Roster full, adding to waitlist
```

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in  
edu.uca.registration.model.CourseTest
```

```
[INFO] Running edu.uca.registration.model.EnrollmentTest
```

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in  
edu.uca.registration.model.EnrollmentTest
```

```
[INFO] Running edu.uca.registration.model.SessionTest
```

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in
edu.uca.registration.model.SessionTest

[INFO] Running edu.uca.registration.model.StudentTest

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0 s -- in
edu.uca.registration.model.StudentTest

[INFO] Running edu.uca.registration.repo.ConfigManagerTest

[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in
edu.uca.registration.repo.ConfigManagerTest

[INFO]

[INFO] Results:

[INFO]

[INFO] Tests run: 24, Failures: 0, Errors: 0, Skipped: 0

[INFO]

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 2.383 s

[INFO] Finished at: 2025-11-01T20:54:25-05:00

[INFO] -----

Test ID	Level	Class / Function	Input / Scenario	Expected Output / Behavior	Result
UT-01	Unit	Course.addToRoster()	Add 3 students to a course with capacity = 2	2 students in roster, 1 student automatically added to waitlist	Pass
UT-02	Unit	Session.setStudents()	Assign map of students to session	Session returns correct size and contains expected ID	Pass
UT-03	Unit	ConfigManager.getPath()	Load data/test/config.json with mock JSON paths	getPath("students") returns "data/test/students.json"	Pass
CT-01	Component / Integration	Menu.addStudentUI()	Simulate user input "B012345\nTest Student\ntest@uca.edu\n"	Student is added to session map and matches given name	Pass
CT-02	Component / Integration	Menu.save() / Menu.load()	Session with one student & course saved, then reloaded	data/*.json files created, and reloaded session contains same data	Pass
ST-01	System	Full workflow: Add → Enroll → Drop → List	Simulate sequential CLI operations on the session	All intermediate data transitions valid, no crashes, correct printed output	Pass